
Project Handover Document

RobotLab's Advanced Threat Monitoring System
Big Chemistry — Extended Detection & Response (XDR) Platform

Date: 12 June 2026

Contents

1	Current State of the Project	2
1.1	Architecture Overview	2
1.2	What Is Live Today	3
2	First Priority: Stabilise NOMAD Oasis	4
2.1	Why This Comes First	4
3	Unit-Scope Security Hardening	5
3.1	Container Image Scanning	5
3.2	Static Application Security Testing (SAST)	5
3.3	Dynamic Application Security Testing (DAST)	5
3.4	Fuzzing	5
4	Forward Research: A Machine Learning Pipeline for XDR	7
4.1	Motivation	7
4.2	Considerations	7
5	Summary Roadmap	8

CHAPTER 1

Current State of the Project

This document hands over the RobotLab’s Advanced Threat Monitoring System, a full Extended Detection & Response (XDR) platform built for Big Chemistry on AWS `eu-central-1`. It captures the system as it stands today and lays out the recommended order of work for the team taking it forward.

1.1 Architecture Overview

The platform follows a two-VPC connected by VPC Peering, with no Transit Gateway. The control plane (**bc-ctrl**, `10.0.0.0/16`) hosts EC2-only workloads — Wazuh SIEM, Splunk SOAR, MISP, and the self-hosted GitHub runner. The production plane (**bc-prd**, `10.30.0.0/16`) runs the EKS cluster carrying the security stack and sensor DaemonSets (Cilium, Falco, Tetragon, Zeek, Suricata, plus the NOMAD Oasis observability target).

The diagram below reflects the current, deployed state of the infrastructure.

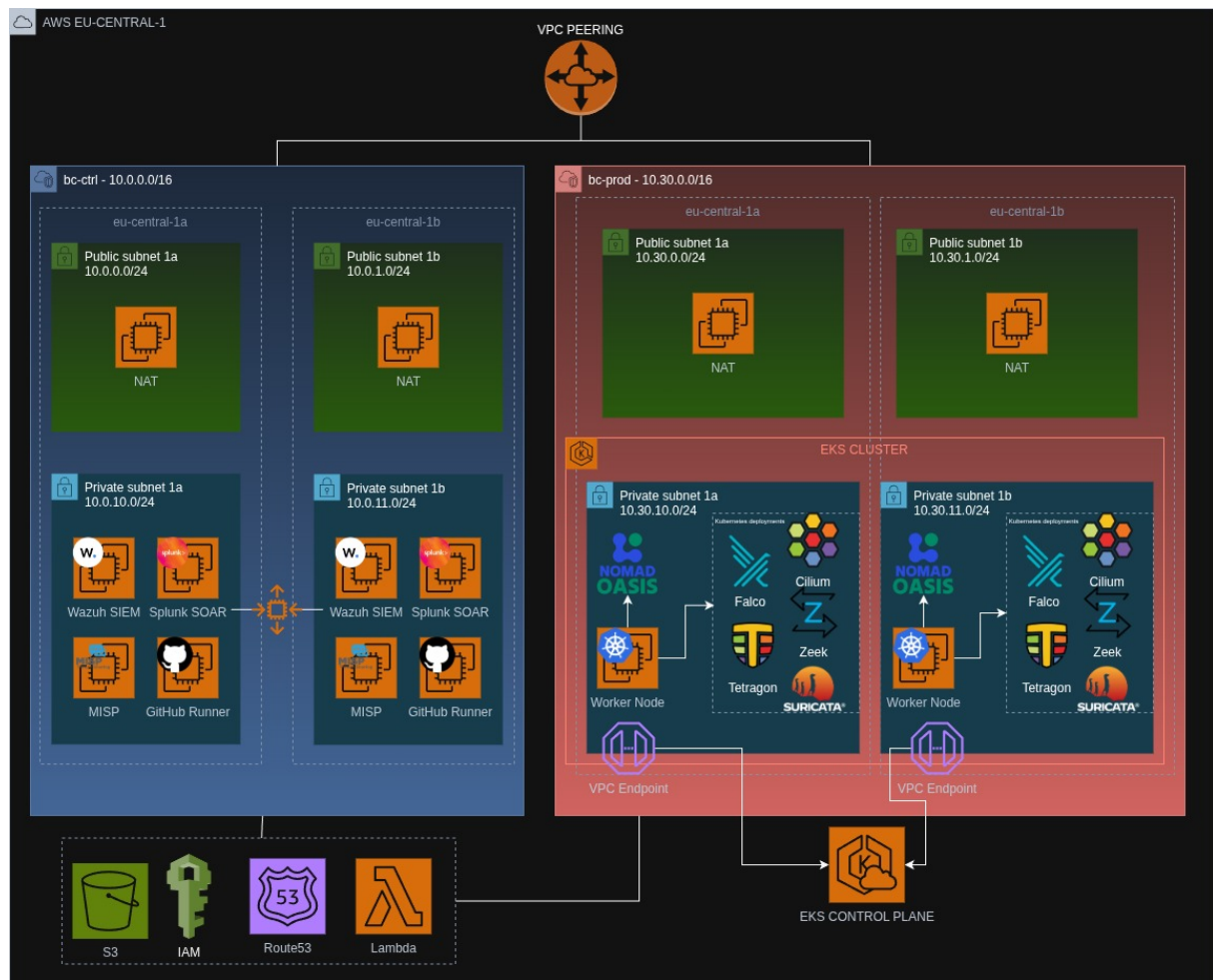


Figure 1.1: Current-state architecture of the XDR platform on AWS eu-central-1. bc-ctrl (the “Brain”) and bc-prd (the “Data”) VPCs are joined by VPC Peering; shared services (S3, IAM, Route53, Lambda) sit beneath both.

1.2 What Is Live Today

- **bc-ctrl (Control plane):** Wazuh all-in-one (Manager + Indexer + Dashboard), MISP threat-intel platform, Splunk SOAR, and a self-hosted GitHub Actions runner.
- **bc-prd (Production plane):** EKS cluster running Cilium, Falco (`modern_ebpf`), Tetragon, Hubble, and the Zeek / Suricata / wazuh-agent DaemonSets.
- **Telemetry pipeline:** Suricata, Zeek, Falco, and Tetragon emit to host-path logs that the wazuh-agent ships to the Wazuh manager.
- **Threat-intel sync:** MISP IOCs are synced hourly into Zeek Intel and Suricata rule formats via sidecar containers.

The platform is operational but remains in a pre-production hardening phase. The chapters that follow define the recommended sequence of work.

CHAPTER 2

First Priority: Stabilise NOMAD Oasis

2.1 Why This Comes First

The single most important first task for the incoming team is to ensure that **NOMAD Oasis is running properly**. NOMAD Oasis is the primary observability and workload target on the bc-prd EKS cluster — it is what exercises the security stack end-to-end. Until it is reliably healthy, every downstream detection, enforcement, and analytics effort is being validated against an unstable base.

To date, NOMAD Oasis has converged on the cluster but has *not* reached a fully validated, repeatable healthy state.

Exit criterion: A documented, reproducible cold-start that brings NOMAD Oasis to a fully healthy state (including browser login) with no manual steps. Only once this is met should the team move on to the hardening work below.

CHAPTER 3

Unit-Scope Security Hardening

With NOMAD Oasis stable, the next focus shifts to a more **unitary scope**: embedding security controls at the level of individual code and container artifacts, rather than only at the runtime / network layer where the platform is already strong. These controls close the supply-chain and application-security gaps that the current detection-centric posture does not cover.

3.1 Container Image Scanning

The platform currently has no container image scanning or admission control, so images are deployed without provenance or vulnerability verification. Recommended measures:

- **Vulnerability scanning** of every image at build time and in the registry (e.g. Trivy or Grype against ECR), failing the pipeline on critical / high CVEs.
- **Image signing and verification** (e.g. Cosign / Sigstore) so only signed, attested images can run.
- **Admission control** on the cluster (e.g. a policy controller) to enforce signature and scan-result requirements before a pod is admitted.

3.2 Static Application Security Testing (SAST)

Introduce SAST into the CI pipeline to catch vulnerabilities in source before they ship — insecure patterns, injection flaws, hardcoded secrets, and unsafe dependencies. Run it on every pull request and gate merges on the results. This complements the existing infrastructure review (`/tf-review`, `/security-audit`) with application-code coverage.

3.3 Dynamic Application Security Testing (DAST)

Add DAST against running services (the NOMAD Oasis web tier, the Wazuh / MISP dashboards, Splunk SOAR, and any exposed API). DAST exercises the deployed application from the outside to surface authentication weaknesses, injection, and misconfiguration that only manifest at runtime.

3.4 Fuzzing

For the higher-risk parsing and ingestion surfaces — the telemetry pipeline that consumes Suri-cata `eve.json`, Zeek logs, and MISP-sourced rules — introduce **fuzz testing** to harden input handling against malformed or adversarial data. Untrusted external data crossing a system boundary is exactly where fuzzing yields the most value.

Goal: shift security “left” to the individual code and container unit, covering SAST, DAST, and fuzzing in CI plus image scanning and signing at the registry / admission boundary — closing the supply-chain gaps the runtime stack cannot.

CHAPTER 4

Forward Research: A Machine Learning Pipeline for XDR

The next strategic direction for the research effort is to build a **Machine Learning pipeline for XDR**. The platform already aggregates a rich, multi-source telemetry stream — Suricata, Zeek, Falco, Tetragon, Wazuh alerts, VPC flow logs, CloudTrail, and GuardDuty — which is precisely the high-volume, labelled-and-unlabelled data foundation an ML detection layer needs.

4.1 Motivation

Today the system is rule- and signature-driven: it detects what it has been explicitly told to look for. An ML layer adds the ability to surface *unknown* threats — anomalous process trees, unusual network flows, credential-misuse patterns, and slow-burn lateral movement — that evade static rules. It also offers a path to reducing alert fatigue by scoring and correlating the existing detections.

4.2 Considerations

- **Cost discipline.** ML infrastructure (training compute, storage, inference) must be scoped and justified.
- **Explainability.** Detections that drive response must be explainable to stakeholders; favour models and scoring whose output a human can reason about during incident response.
- **Data governance.** Telemetry includes sensitive operational detail; apply the same least-privilege and KMS-encryption discipline used across the rest of the platform to the data lake and ML artifacts.

Direction: evolve the monitoring system from a rule-driven detection platform into a learning system — an ML pipeline that turns the existing multi-sensor telemetry into adaptive, anomaly-aware XDR detection.

CHAPTER 5

Summary Roadmap

Order	Workstream	Outcome
1	Stabilise NOMAD Oasis	Reproducible, fully healthy cold-start including browser login; stable base for all downstream work.
2	Unit-scope security hardening	Container scanning + signing + admission control, and SAST / DAST / fuzzing in CI.
3	ML pipeline for XDR	Adaptive, anomaly-aware detection built on the existing multi-sensor telemetry.

Table 5.1: Recommended order of work for the incoming team.

End of Handover Document.